A Comprehensive Study of YOLOv8, YOLOv11, and YOLOv26 on pothole segmentation

Ulyses Ortega
Richard Yam-Ciau
Duc Nguyen
https://github.com/ulyses97/Official-Deep-Learning-Final

Abstract

Potholes are a common form of road damage that can create safety risks, increase vehicle maintenance costs, and reduce transportation efficiency [8]. Automated pothole detection can support not only road inspection systems but also the average commuter by identifying damaged road regions more quickly than manual inspection. While object detection can locate potholes using bounding boxes, instance segmentation provides a more detailed image by outlining the visible pothole region [12]. This is useful because potholes vary in shape and size. In this project, we compared three YOLO-based segmentation models: YOLOv8n-seg [1], YOLO11n-seg [11], and YOLO26n-seg [18]. Each model had its own pretrained weights and was fine-tuned on the same pothole segmentation dataset. To ensure a fair comparison, each model was trained using the same dataset, evaluation process, and general experimental setup, while selected hyperparameters were tuned sequentially instead of a wide grid search. The hyperparameters include learning rate, image size, and weight decay. Model performance is evaluated using segmentation metrics such as precision, recall, mAP50, and mAP50-95, with mAP50-95 used as the primary measure of segmentation quality [13]. Results on unseen test images told us that YOLOv8 provided high mAP50-95 along with YOLO26, with YOLO11 being right behind them. This tells us that changing parameters is very important, such as image size. Changing the image size is what allowed YOLOv8 to do better than the other test results because the model did not need to see a lot of detail, while the others did. While YOLOv8 did show a higher mAP50-95, being at 0.469, YOLO26 almost achieved that number, being at 0.464, but with a higher image size, which means it was able to handle more detail. These findings tells use that it is important to see what parameters each model has and what limitations each model has. Many low-end models may perform better in low parameters, but when higher parameters are inputted the higher-end models may outperform the low-end models.

A. Introduction

There are many different YOLO models, and each one has a different architecture, which has become very useful in today's modern society. Potholes are an inconvenience to anyone when driving on a road and can cause either damage to one's vehicle or even prevent one from going down their intended route. YOLO can be used to detect potholes by using object detection, but it can also use instance segmentation. Now, object detection is not bad and does the job well in detecting objects with a bounding box, but segmentation is better at estimating a size, which can tell us how big a pothole will be and its area. Ultralytics tells us that instance segmentation " is a method that treats each object as a unique entity"[1]. This tells us that instance segmentation will take any object that we want to use or detect will identify and outlined, the exact sample of the object. For us, we will be looking at three different models of YOLO, in which they are YOLOv8, YOLO11, and YOLO26. We will be changing the parameters of each YOLO model and see what parameters give us the best results on segmentation. We used an Nvidia 2070 Max Q design GPU and also an NVIDIA RTX A1000 to run our test on each YOLO model. We will be comparing the architecture of the models and see what makes each model improve better than the other.

A.1. Main Contributions

1. We fine-tune YOLOv8n-seg, YOLO11n-seg, and YOLO26n-seg on the same pothole segmentation dataset.

2. We compared the models under a controlled experimental setup using the same data and evaluation metrics.

3. We analyze the effects of key parameters, including learning rate, image size, and weight decay.

4. We evaluate model performance using segmentation mAP50–95 as the primary metric because it better reflects mask quality across multiple IoU thresholds.

5. We test the best performing configurations on unseen images to evaluate generalization.

B. Related Work

B.1. Convolutional Neural Networks

In our deep learning and computer vision courses, we covered the core concept of Convolutional Neural Networks (CNNs), which are the main building blocks in computer vision. A CNN is designed to process images by learning visual patterns through training. These patterns can include simple features such as edges and corners, and can even include more complex features such as the objects' shapes or surface textures. The main benefit for the use of using CNNs on images is that they can focus on small regions of an image at a time. Instead of treating an image as unrelated pixels, a CNN keeps the spatial structure of the image, meaning the CNN. learns how its nearby pixels are related to each other. This is important for pothole segmentation because a pothole could be identified by its boundary, rough texture, dark center, or contrast with its surrounding road. As we studied in our courses, a simple convolutional operation can be written as:

$$Y = \sigma(X * W + b)$$

In this question, X is the input image or feature map, W is the learned filter (can include weights), b is the bias, * is the convolution operation, and is the activation function. In simpler terms, the model slides a small filter across the image and learns which visual patterns are useful.

Relating to our project on pothole detection, earlier CNN layers normally learn simple patterns such as edges, color changes, and shadows. Deeper layers combine these simple features into more meaningful patterns such as cracks, pothole borders, and even be able to read damaged regions. This layered structure helps the model tackle simple and higher-level tasks.

B.2. YOLO-Based Computer Vision Models

YOLO stands for You Only Look Once. It is a group of computer vision models made for making quick predictions [1], [11]. YOLO models are often used in real time tasks since they are designed to process an image in a single forward pass. This makes them useful for applications such as traffic monitoring, surveillance, autonomous driving and road inspection. Most modern YOLO models are composed of three main parts: a backbone, a neck, and a head [1], [6]. The backbone extracts helpful visual features from the input image. The neck combines information from different feature levels so the model can understand the smaller details and the larger objects. The head provides the final prediction. In a detection model, this prediction usually includes a bounding box and class labels. In a segmentation model, the output also includes a mask.

YOLOv8 is a popular Ultralytics YOLO model that supports various tasks such as detection and segmentation [1]. The model uses an efficient architecture and an anchor-free prediction method. In this case, Anchor-free means that the model does not rely as much on predefined box shapes. This can help when potholes vary in all shapes and sizes.

YOLO11 is an updated version in the Ultralystics YOLO family [11] compared to its predecessors, aiming to improve efficiency and accuracy. YOLO26 is also included in this project as a newer model option [8]. By comparing YOLOv8n-seg, YOLO11n-seg, and YOLO26n-seg, we can investigate whether newer YOLO models offer better pothole segmentation results under the same training conditions.

YOLO26 is the latest in the YOLO line of real-time object detectors for edge and low-power devices[18]. YOLO26 has become faster, lighter, and easier to deploy in real-life systems [18]. YOLO26 can achieve a higher accuracy on small objects, which is perfect for us since potholes are of different sizes. Also, YOLO26 is one of the most practical and deployable YOLO models to date [18].

B.3. Object Detection Versus Instance Segmentation

Object detection and instance segmentation function similarly but they do produce different types of output. For object detection, an object of interest receives a rectangular box around it. This bounding box is useful when the main goal is to just locate the object. However, for potholes, a bounding box may not be fully accurate since it can contain things outside of the actual pothole.

Instance segmentation provides results with more details by predicting a mask for each object [12]. Using the mask allows the model to attempt to identify the exact pixels that belong to the object. This is highly beneficial for potholes because potholes often have irregular boundaries. A segmentation mask can provide a better capture of the true shape of the damaged area.

Our project focuses on segmentation instead of only detection. A bounding box can provide information on where it is located but a segmentation mask can answer what part of the image is the pothole.

B.4. Pothole Detection with Deep Learning

Even with advanced technology in this day and age, pothole detection still remains challenging mainly because potholes can look very different from one another. Some potholes can look large and dark, while others appear to be shallow and blend into the roads. Pothole detection can

struggle harder when shadows, cracks, lighting and terrible repairs are introduced into its detection. These factors can cause the model to flag false positives, the model detects a pothole even when there is not one present. The model can also flag false negatives as well, where the model misses a real pothole.

Deep learning is useful for this problem because it can learn patterns from many datasets instead of relying only on hand made rules. The only caveat is that performance depends on the quality of the dataset, the model's architecture, and the training setup. One model could work well on one road surface but it does not guarantee it is going to perform the same on a different road with different lighting or pavement textures. For this reason, it is important to test the models on unseen data and observe its generalizability.

C. Dataset and Experimental Setup

Table 1. Summary of the dataset used in this project.

Dataset	**Images / classes**	**Task**	**Models**
RoboFlow Roadvis Seg	3,174 images / 1 class: pothole	Instance segmentation	YOLOv8n-seg, YOLO11n-seg, YOLO26n-seg

C.1. Dataset Description

The dataset our group used in this project is the Roadvis pothole segmentation dataset from RoboFlow [9]. This dataset contains 3,174 images and uses one class label: pothole [9]. The main reason why our group decided to proceed with a one-class dataset is because the goal of the project is to focus only on pothole segmentation, not general road damage classification.

Using one class keeps the training simple and easier to understand. Road damage can include cracks, patches, rough asphalt, and other defects. Our model may struggle dealing with these types of road damage which can end up looking similar to potholes. By only focusing on one pothole class, the models can learn the specific visual features of potholes.

As for our dataset, each image includes segmentation labels that outline the pothole region.

These labels are used during training so the model can learn which pixels belong to the pothole class. All three YOLO models will have its own pretrained "default" version. Each model will be trained and evaluated using the same dataset, the same tuning budget and the same hyperparameter tuning to keep the comparison fair.

C.2. Dataset Challenges

Even though the dataset contained various images and proper labels, the dataset still includes several challenges that makes pothole segmentation challenging. A few issues are that potholes vary in size, some are large and easy to see, while others are small and blend into the road. Another issue is potholes have irregular shapes, compared to common objects such as cars or signs, potholes are not consistent in shape/outline. Another big issue is the roads themselves, they can be visually noisy, some roads can have cracks, shadows, water stains, and terrible asphalt repairs can look similar to potholes.

Even in today's modern technology, a model can struggle dealing with discrepancies in lighting , a pothole can appear completely different in daylight compared to nighttime. Strong shadows can make a normal part of the road be part of the pothole while weak contrast can make a real pothole harder to detect. All these challenges are the reason why segmentation quality is important. The segmentation model's main objective is to not only detect potholes but also draw the mask accurately to match their boundaries.

C.3. Hardware and Software Environment

The experiments were run using the groups available GPU hardware. The tuning/testing was operated using two devices, in order to increase efficiency. One system used was the NVIDIA 2070 Max-Q Super GPU and the second system was NVIDIA RTX A1000

The hardware setup is important because the segmentation models can be very large and require large amounts of GPU memory. Larger image size can improve mask quality, but they also require more computation and memory. This is the reason why our group limited the batch size, epochs, and the amount of hyperparameters tuned, since this affects how many experiments can be run and how long training takes.

D. Methodology

D.1. Overview of Experimental Pipeline

The main methodology of this project is to compare the three YOLO segmentation models under similar conditions. Each model starts with its own pretrained "default" weights, then it will be fine tuned on the pothole dataset, and finally be evaluated using the same metrics. This allows us to observe the results and how the models compare with each other without changing the overall experimental process for each one.

Each model will undergo the following pipeline:

1. Prepare the pothole segmentation dataset
2. Select YOLO-seg model
3. Load pretrained weights
4. Train model on the training set

5. Evaluate the model on the validation set
6. Tune learning rate, weight decay, and image size
7. Select the best configuration using segmentation mAP50–95
8. Test the best configuration on unseen images
9. Compare the models using both metrics and visual predictions

The end results will allow us to compare both the model architectures and how each model responds to hyperparameter tuning.

D.2. YOLO Training Methodology

For our method in analyzing the best YOLO model for segmentation, we will be using the same dataset for all three models. The dataset is from Roboflow, which uses 3174 images and only has one class. It was important to only have one class because we are not focusing on road damage. Also, the model can later on become confused and not know how to classify some images where there are multiple potholes or cracks that may look like potholes. So we kept it to one class, with it only being potholes, so our model does not get confused. The three models that we will be looking at are YOLOv8n-seg, YOLO11-seg, and YOLO26-N-seg. In order to keep a fair comparison, we tried to keep everything the same for each model. The same number of epochs, settings, dataset, and also the same parameters were used. The parameters we set to change were the learning rate, image size, and also the weight decay. We then compared all the data onto an Excel sheet and also made plots for each run. Then tested the best model, which looked at the best segmentation for mAP50-95.The reason we look at mAP50-95 is that we are looking at segmentation, and mAP50-95 tells us how well the model is at making correct predictions while ignoring false detections. While there will be other values that are given, we will look more at seg-mAP50-95 because this tells us the segmentation quality, and the other values can help us explain why the other models either performed better or worse. Also, the other metrics, such as mAP50, precision, loss, and recall, all describe one part of the model, while mAP50-95 gives us the best overall segmentation accuracy.

D.3. Models Evaluated

The three YOLO segmentation models were used in this project. The following section will have a short description of each model used.

YOLOv8n-seg: YOLOv8n-seg is the nano segmentation version of YOLOv8 [1]. This model is lightweight and designed to be efficient. Since this model is a smaller variation of the other YOLO models, it can be trained faster and it is easier to run on limited hardware. With this in mind, a smaller model may not always produce the most accurate segmentation masks.

YOLO11n-seg: YOLO11n-seg is the nano variant of the YOLO11 versions[11]. This model was designed to improve real-time detection and segmentation through updated model design and more efficient feature extraction. For our project, this model predicts both the location of the pothole and the masks that outline its shape. With this improved feature, it will help us compare whether a newer YOLO version improves pothole segmentation quality.

YOLO26n-seg: YOLO26n-seg is the newest model in this comparison. YOLO26 is expected to include newer design and training improvements [8] compared to YOLOv8 and YOLO11. YOLO26 design makes it less complex while also allowing targeted innocations to deliver faster, lighter, and more easier for deployment [18].

E. YOLO Model Comparison

E.1. YOLOv8 Intro

On the Ultralytics website, it states that "YOLOv8 was released on January 10, 2023, and offered cutting-edge performance in terms of accuracy and speed" [1]. Some features that YOLOv8 has are an advanced backbone and neck architecture, resulting in improved feature extraction and object detection performance"[1]. YOLOv8 also finds a balance between accuracy and speed, so it is practical to be used in detecting objects in real time. Reading more into the YOLOv8 architecture, it says that it uses the C2f module, which is a cross-stage partial bottleneck with two convolutions, and this helps "improve gradient flow and allows the network to learn richer feature representations without heavily taxing GPU memory"[2].

E.2. YOLOv8 Architecture

Taking a look at the architecture, YOLOv8 has 3 sections: the backbone, neck, and head. The backbone uses a CNN that has been optimized for speed and accuracy. The backbone then looks at the image and sees important visual features such as shapes, shadows, or edges, and extracts them. The neck then "refines and fuses the multi-scale features extracted by the backbone" [4]. What this means is the neck takes the information from the backbone and combines it, which makes the model understand the image much better. The head finally gives its final predictions, which can include a bounding box, object confidence scores, and class label[4]. What YOLOv8 has is an anchor-free approach to bounding box prediction, which is something that was added, whereas the previous YOLOs used an anchor-based method[4]. This anchor-free approach that YOLOv8 has predicts where the location of the object is, without using the present box shapes that previous YOLO models used.

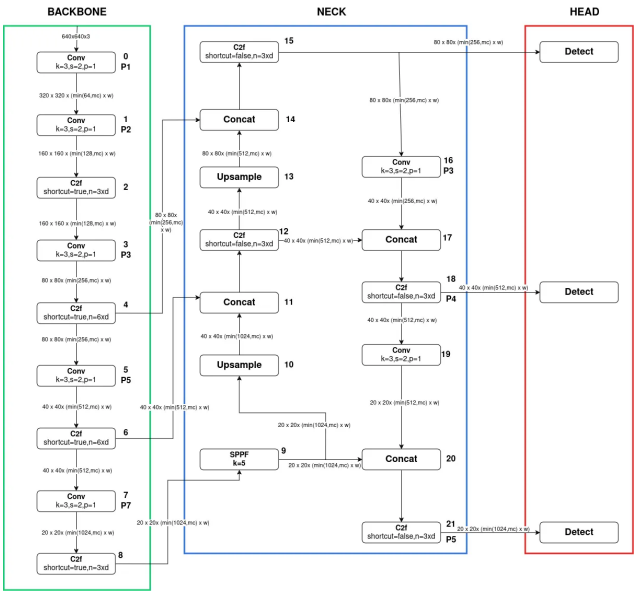

Figure 1. YOLOv8 architecture from [3].

This is useful for this experiment because potholes are usually irregular shapes, with some being very big or small, and some being round or others being cracks. It depends less on preset boxes and more on learning object locations more freely. Using these architectures, YOLOv8 improves on object detection from its predecessors and improves on accuracy and speed, becoming very reliable in real-world applications.

E.3. YOLO11 Intro

For the second model, we trained and evaluated YOLOv11-seg, specifically the nano segmentation version, yolo11n-seg.pt. YOLOv11 is part of the Ultralyrics YOLO family and is designed to improve real-time object detection and segmentation through more efficient feature extraction and model design [11]. In this project, YOLOv11-seg was used for pothole instance segmentation; the model predicts where a pothole is located, as well as the pixel-level mask that outlines the pothole shape. This is useful because potholes are often irregular, uneven, and non-rectangular. Compared to its default version, bounding boxes can show the approximate location of a pothole, but a segmentation mask provides more useful information about its actual boundary and is [12].

E.4. YOLO11 Architecture

For the Architecture of YOLO11, it has an improved backbone and neck along with C3k2 blocks, SPPf, C2PSA, and an attention mechanism. It is said that "YOLO11 extends and enhances the foundation laid by YOLOv8, introducing architectural innovations and parameter optimizations to achieve superior detection performances"[17]. With the improved backbone, YOLO11 is better at detecting edges, cracks, shadows, and rough textures. YOLO11 also has SPPF, and this is spatial pyramid pooling-fast, which is said to be in the previous version, but also includes a new cross-stage partial with spatial attention or C2PSA. What this does is improve how the model can focus on important regions in an image[17]. When pooling features spatially, this helps the C2PSA block and enables YOLO11 to focus on important areas, which can improve the detection accuracy for objects of different sizes or in different positions. [17]. YOLOv8 had the C2f block, but in YOLO11, it is replaced with the C3k2 block, and this block is faster and more effective, which enhances the overall performance of the feature aggregation process. Another addition to YOLO11 is the attention mechanism, and what this does it is it allows the model to focus on important key regions in an image, which can improve detection. This can help with objects that are small or even objects that are occluded [17].

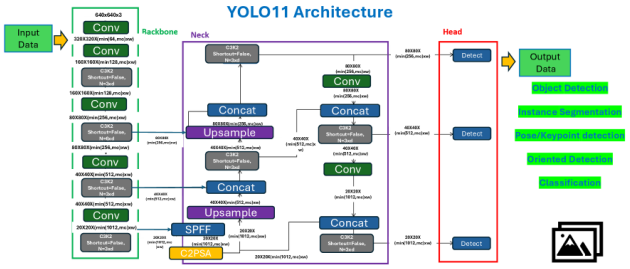

Figure 2. YOLO11 architecture diagram.

E.5. YOLO26 Intro

YOLO26 is the latest generation in the Ultralytics YOLO family, released in 2025 as a real-time object detector designed for edge and low-power deployment [18]. Compared to its predecessors, YOLO26 focuses on reducing model complexity while maintaining competitive accuracy, making it faster, lighter, and more practical for real-world systems [18]. The YOLO26n (nano) variant used in this project is the smallest and most efficient version, containing approximately 3.05 million parameters and requiring only 10.2 GFLOPs per inference pass.

YOLO26 introduces several architectural improvements over YOLO11, including an updated C3k2 block design, a revised Segment26 prediction head, and a new MuSGD optimizer that is trained natively with the pretrained weights[18]. These changes are designed to improve gradient flow and feature reuse while keeping the model compact. For this project, YOLO26n-seg was selected as the third comparison model to evaluate whether its newer design improvements translate to better pothole segmentation performance under the same training conditions as

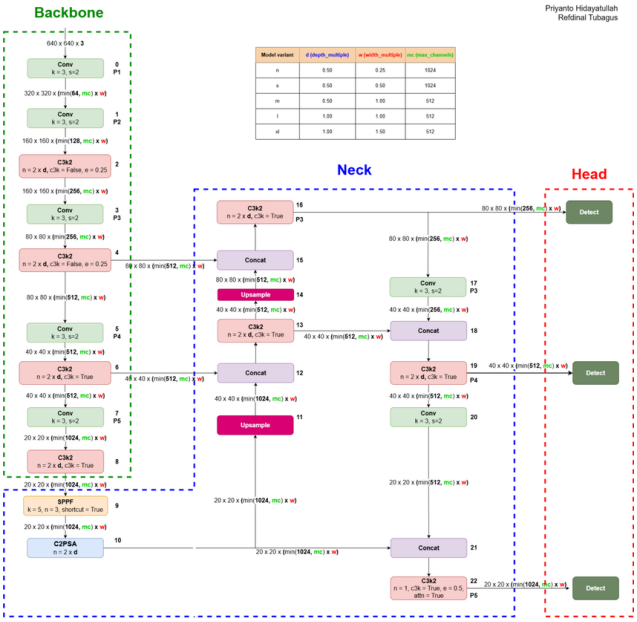

Figure 3. YOLO26 architecture diagram.

YOLOv8n-seg and YOLO11n-seg.

E.6. YOLO26 Architecture

YOLO26n-seg follows the same three-part structure as other YOLO models: a backbone, a neck, and a segmentation head. However, several internal components have been updated compared to YOLO11.

The backbone of YOLO26n uses a series of Conv layers followed by C3k2 blocks, which are cross-stage partial blocks with two convolutions. These blocks improve gradient flow by allowing the network to learn richer feature representations without heavily increasing GPU memory usage. The backbone also includes a Spatial Pyramid Pooling Fast (SPPF) block, which captures multi-scale contextual information from the image. Following SPPF, a C2PSA (Cross-Stage Partial with Spatial Attention) block further refines the features by allowing the model to focus on the most informative spatial regions.

The neck uses a feature pyramid structure with upsampling and Concat layers to combine low-level and high-level features across different scales. This multi-scale feature fusion helps the model handle potholes of varying sizes — from small surface cracks to large damaged road areas.

The head of YOLO26n-seg uses the Segment26 module, which is a revised segmentation head that predicts both bounding boxes and pixel-level instance masks. The Segment26 head outputs mask coefficients and prototype masks, which are combined to produce the final segmentation output for each detected instance. The full YOLO26n-seg model contains 309 layers and 3,053,055 parameters at 10.2 GFLOPs, making it comparable in size to YOLO11n-seg while introducing updated internal components.

A key difference in YOLO26 is its native training optimizer, MuSGD. The pretrained weights provided by Ultralytics were originally trained using MuSGD with a higher learning rate (lr0=0.0054) and momentum of 0.947. As shown in our tuning experiments, using `optimizer=auto` to invoke MuSGD during fine-tuning consistently outperformed AdamW, suggesting that the pretrained features are better preserved when the same optimizer family is used.

F. Training and Tuning

F.1. Use of Pretrained Weights

All three models used pretrained weights [1], [8]. [11] before we started the fine tuning. Due to time limitations, pretrained weights are useful because the model has already learned general image features from a large dataset. For example the model may already understand basic shapes, edges, and textures . The fine tuning adapts this general knowledge for our pothole dataset. We went with this realistic approach because training a model from scratch will require a large amount of data and expensive computational power. Since our project works with a smaller pothole dataset, the pretrained weights are more than enough to help the models learn more effectively.

F.2. Fair Comparison Rules

In order to make the comparison fair for each model, the same dataset and evaluation process will be conducted. Each model is trained on the same dataset, tuned with the same hyperparameter values, and evaluated with the same metrics. We will try our hardest to follow these rules so all comparisons can be fair but might have to change a couple of things. The main differences are the YOLO model versions themselves and their pretrained weights + biases. This setup helps ensure that the difference in performance is due to the models architecture and processes rather than different dataset or evaluation methods.

G. Hyperparameter Tuning Strategy

G.1. Importance of Hyperparameter Tuning

Hyperparameters are settings that govern how a model trains. They are different from model weights, which are learned automatically. The hyperparameters and values were chosen by the suggestions from OpenAI ChatGPT, with the main purpose of realistic and impactful hyperparameters. For this project, the main hyperparameters chosen are learning rate, image size, and weight decay [14]. These settings can strongly affect performance. A model with poor hyperparameters may train slowly, could overfit

the dataset, or even produce inaccurate masks. Since this project compares the three YOLO models, hyperparameter tuning is important to give each model a fair chance to perform well.

G.2. Learning Rate Tuning

When looking at which parameters, we looked at the learning rate, which controls how fast the model learns. A simplified update rule is:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t) \tag{1}$$

In this equation, t represents the model weights at training step t, is the learning rate, and L is the loss function. In other words, the learning rate controls how quickly the model updates its weights during training.

Picking the right learning rate is important because if we were to pick a learning rate that is too high, the model can change too aggressively and become unstable, producing bad results. This can produce poor masks or inconsistent results. If the learning rate is too low, the model may learn too slowly and will not improve at all within the training time. So, picking the right learning rate is very important, so we will be using different learning rates. Each model tested

At the start of this project, we started with a baseline, where we changed the learning rate three times and then saw which learning rate was the best. We repeated the same process with weight decay and image size while keeping the best learning rate.

1. Learning rate values: 0.001, 0.0001, and 0.0003
2. Image size values: 512, 640, and 800
3. Weight decay values: 0.0001, 0.0005, and 0.001

G.3. Weight Decay Tuning

Weight decay is a regularization technique. This hyperparameter controls how much the weights are restricted, which helps combat overfitting by keeping the weights small, this forces the model to learn smoother, simpler, and more generalizable patterns. Overfitting happens when a model learns the training data too closely and performance begins to drop. For segmentation, overfitting can cause a model to perform well on similar looking potholes but fail on unseen road images.

A simple loss function with weight decay is:

$$L_{\text{total}} = L_{\text{box}} + L_{\text{cls}} + L_{\text{mask}} + \lambda \|\theta\|_2^2 \tag{2}$$

In this equation, L_{box} is the bounding box loss, L_{cls} is the classification loss, and L_{mask} is the segmentation mask loss. The last term, $\lambda\|\theta\|_2^2$, is the weight decay penalty. In this case, the penalty discourages the YOLO models from relying too much on weight values.

Weight decay is important for generalization, but too large of a weight decay can hurt performance. In our project, we had to determine the best value while still retaining the models' ability to be flexible in learning irregular boundaries.

G.4. Image Sizing Tuning

The next hyperparameter we started tuning was image size. Image size is important for segmentation because masks depend on visual information. A larger image size can help the model see the pothole edges more clearly. This can improve segmentation quality, especially when potholes appear small or have uneven boundaries. The only trade-off about using larger images is that it requires more computation. In our project, our image size is represented as:

$$\frac{\text{New Pixel Size}}{\text{Old Pixel Size}} \tag{3}$$

Increasing the image size increases the approximate pixel cost.

Increasing image size may improve accuracy, but it also increases training cost.

G.5. Model Selection Metric

The main metric used in the project to select the best model is segmentation mAP50-95 [13]. This metric measures how well the model predicted mask overlaps with the correct true mask across several overlap thresholds. For this metric we want to observe a higher score, which will show the model is producing accurate segmentation results. We will also be using precision and recall as metrics for this project. Precision measures how many predicted potholes are correct. Recall measures how many real potholes the model can detect/locate. For these metrics, we want to see high scores which will demonstrate the models ability to make fewer false detections and miss fewer real potholes. For this project, mAP50-95 is emphasized because it gives a stricter measurement of segmentation quality.

H. Results

H.1. YOLOv8 Training Results

The table here gives a summary of all the training done in this experiment, which includes all the parameters that were changed, such as the learning rate, weight decay, and image sizing. As explained before, we will be looking at the mAP50-95 values because this gives us the validation segmentation, and what is being discussed is the segmentation quality. For the learning rate 3 rates were tested, in which we saw that when the learning rate was at 0.0003, it

performed high in mAP50-95 compared to our other learning rate values. We saw that at 0.001, the learning rate didn't perform as well as at 0.0003 because the model may have been too aggressive. Since it was being too aggressive to learn, the model could have missed the best solution training. As for our learning rate at 0.0001, we saw a drop here, being at 0.3798, which was the worst result of all the learning rates. The reason it performed badly was that the model may have been learning too slowly, and with only 10 epochs, it could not improve. When testing at 0.0003, it provided a good balance between the rates and saw an improvement in our mAP50-95, which was at 0.4317, which is not a huge jump from our learning rate at 0.001, which was 0.4238. But we did see an improvement in the final learning rate tuning results.

Table 2. YOLOv8 training results for different hyperparameter settings.

Run	LR	Image Size	Weight Decay	seg_mAP50–95
V8_E00	0.001	640	0.0001	0.3596
V8_E01	0.001	640	0.0001	0.4239
V8_E02	0.0001	640	0.0001	0.3799
V8_E03	0.0003	640	0.0001	0.4318
V8_E04	0.0003	512	0.0001	**0.4516**
V8_E05	0.0003	800	0.0001	0.4376
V8_E06	0.0003	800	0.0005	0.4394
V8_E07	0.0003	800	0.001	0.4346

H.2. Image Size

For our image size, we changed it to 640, 512, and 800, where we kept the learning rate the same since we want to have a fair comparison. We look at the image size, which helps control the resolution of the images used in training. The bigger the image size, the more detail an image has, which can help detect small potholes or also have better mask boundaries. Increasing the image does have it's draw backs, where the training can be harder and increase computations. We assume that the higher image size will perform better, but this is not the case here. At 512 image size, the mAP-95 had a value of 0.4516, which is higher than our other image size rates. The reason for this is that the image size gave enough information for the model to be stable. An image size of 800, which is very big, could have been more complex, and also, the model we are using is YOLOv8 nano. We did see that 512 performed better than 800 we decided to go with 800 on our image size to test our weight decay, because some potholes are difficult to see, and having greater detail, such as 800, will help with those details.

H.3. Weight Decay

As for the final parameter changed, we looked at weight decay, which helps our model not overfit by not allowing our model not memorize the training data. Now knowing that we saw that at 0.0005, we performed a little bit better than. This tells us that the lowest of the weights decayed performed better because we were not restricting our model too much. If we were to lower our weight decay too much, our model would have a lot of freedom and would begin to overfit, which would make our model look for patterns that it is already used to. So if we were to try a different dataset and have a low weight decay, the model will perform poorly. That is why, with a balanced weight decay, we have restricted it but not too much so it can learn.

H.4. YOLO11n-seg Tuning Results

H.5. Learning Rates

Moving on to the learning rate, we used the same values we used for YOLOv8, and here we saw that our learning rate of 0.0003 performed better than our other learning rates. Here we saw that at 0.0003, our model has a mAP50-95 of 0.4346. Even tho at 0.001we saw an increase, which was better only by a little, we decided to go with 0.0003 because it is a better balance for our model.

H.6. Image Size

The image size was changed, and there was an increase in our mAP50. What the data show is that at 800 for the image size, the mAP50-95 was at 0.4563. With the largest image size, the model performed better than when it was at 512. This tells us that YOLOv11 is able to handle large image sizes. The improved architecture that YOLOv11 has may have helped with being able to handle large image sizes.

H.7. Weight Decay

Moving on to our last parameter, we looked at weight decay. The best weight decay that our model had was at 0.0001, which gave us a mAP50-95 of 0.4563, which tells us that the smaller weight decay allowed the model to learn without being restricted by a lot. The higher weight decays, like at 0.0005, had lower values because they may have prevented the model from learning difficult potholes that are more difficult to detect.

H.8. YOLO26n-seg Tuning Results

Table 3 summarizes the first phase of YOLO26n-seg tuning. In this phase, learning rate, image size, and weight decay were adjusted while tracking segmentation mAP50–95 as the main evaluation metric.

Table 3. YOLO26n-seg tuning results.

Run	Change	lr0	imgsz	WD	seg_mAP50–95
E01_baseline	Baseline	1×10^{-3}	640	1×10^{-4}	0.420
E02_lr1e4	Low LR	1×10^{-4}	640	1×10^{-4}	0.381
E03_lr3e4	Mid LR	3×10^{-4}	640	1×10^{-4}	0.402
E04_imgsz512	Small image	3×10^{-4}	512	1×10^{-4}	0.397
E05_imgsz800	**Large image**	$\mathbf{3 \times 10^{-4}}$	**800**	$\mathbf{1 \times 10^{-4}}$	**0.437**
E06_wd5e4	Mid WD	3×10^{-4}	640	5×10^{-4}	0.421
E07_wd1e3	High WD	3×10^{-4}	640	1×10^{-3}	0.402

As shown in Table 3, the best result from the first tuning phase was E05_imgsz800, which used an image size of 800 and achieved a segmentation mAP50–95 of 0.437.

H.9. Learning rates

Learning rate was evaluated at three values — 1e-3 (E01, baseline), 1e-4 (E02), and 3e-4 (E03) — while keeping all other hyperparameters fixed at imgsz=640 and weight-decay=1e-4. The mid-range value of lr0=3e-4 (E03) achieved the highest precision (0.840) but slightly lower recall compared to the baseline. The default lr0=1e-3 (E01) yielded the best overall seg-mAP50-95 (0.420) in Phase 1, while lr0=1e-4 (E02) converged too slowly within 10 epochs, producing the lowest mAP50-95 (0.381). Based on these results, lr0=3e-4 was selected as the learning rate for subsequent image size and weight decay experiments

H.10. Image Size

Image size was evaluated at 512 (E04), 640 (E01, baseline), and 800 pixels (E05), using lr0=3e-4 and weight-decay=1e-4. Increasing image size to 800 (E05) produced the largest single improvement across all Phase 1 experiments, raising seg-mAP50-95 to 0.437 and recall to 0.717 — the highest recall seen in any Phase 1 run. This suggests higher resolution allows the model to better capture the fine boundary details of pothole masks. Reducing to 512 (E04) slightly degraded performance across all metrics, confirming that resolution is a critical factor for this segmentation task. Due to the increased image size, batch size was reduced from 8 to 4 to fit within the 6 GB VRAM budget of the NVIDIA RTX A1000.

H.11. Weight Decay

Weight decay was evaluated at 1e-4 (E01, baseline), 5e-4 (E06), and 1e-3 (E07), using lr0=3e-4 and imgsz=640. Neither higher value improved over the baseline — E06 matched the baseline mAP50-95 (0.421 vs. 0.420) while E07 dropped to 0.402. This indicates the model does not benefit from stronger L2 regularization at 10-epoch fine-tuning, likely because the pretrained YOLO26n weights are already well-regularized and the pothole dataset (2,980 training images) is large enough to avoid significant overfitting. The default weight-decay=1e-4 was retained for all subsequent experiments.

H.12. Official Recipe and Advanced Settings

Table 4 shows the second phase of YOLO26n-seg tuning, where official recipe settings and advanced training options were tested.

Table 4. YOLO26n-seg additional tuning results.

Run	Change	Optimizer	Special Setting	seg_mAP50–95
E08_officialloss	Loss weights	AdamW	box=5.63, cls=0.56	0.427
E09_officialoptimizer	Momentum/WD	AdamW	momentum=0.947	0.409
E10_officialaugment	Augmentation	AdamW	copy_paste=0.075	0.424
E11_musgd	**MuSGD**	**auto**	**lrf=0.0495**	**0.470**
E12_coslr	Cosine LR	AdamW	cos_lr=True	0.449

As shown in Table 4, E11_musgd gave the best overall result, reaching a segmentation mAP50–95 of 0.470. This shows that the native optimizer setting improved the YOLO26n-seg model more than the other advanced settings tested.

Phase 2 experiments (E08–E12) were designed to test hyperparameters from the official YOLO26n training recipe, including loss weights (E08), optimizer momentum and weight decay (E09), copy-paste augmentation (E10), the native MuSGD optimizer (E11), and cosine learning rate decay (E12). All Phase 2 runs used the best image size from Phase 1 (imgsz=800, batch=4). The standout result was E11-musgd, which uses the optimizer=auto setting to invoke the MuSGD optimizer — the same optimizer used to train the official YOLO26n pretrained weights. E11 achieved the highest seg-mAP50-95 of 0.470, a +5.0 point improvement over the baseline. E12 (cosine LR) was the second-best at 0.449, while E09 (official AdamW momentum) underperformed, suggesting MuSGD-specific hyperparameters do not transfer well to AdamW. Based on these results, the E11 configuration was selected for the final 100-epoch training run.

H.13. Final Training

The best configuration from all tuning experiments (E11: optimizer=auto/MuSGD, lr0=5.4e-3, weight-decay=6.4e-4, imgsz=800) was used to train the final model for 10 epochs. The model was evaluated once on the unseen test split (96 images). It achieved seg-mAP50-95 = 0.464 and seg-mAP50 = 0.759, confirming the tuning process produced a well-generalizing model.

I. Final Cross-Model Comparison

I.1. Comparison Between the YOLOs

Table 5 summarizes the best segmentation mAP50–95 achieved by each model after hyperparameter tuning and final training.

Among the three models, YOLO26n-seg achieved the highest segmentation mAP50–95 of 0.464 on the unseen test set, followed closely by YOLO11n-seg and YOLOv8n-seg. All three models are in the nano category with similar

Table 5. Final model comparison on unseen test set.

Model	Best Config	Seg mAP50	Seg mAP50–95	Params
YOLOv8n-seg	lr=3e-4, imgsz=512	0.712	0.452	3.4M
YOLO11n-seg	lr=3e-4, imgsz=800	0.743	0.456	2.9M
YOLO26n-seg	**MuSGD, imgsz=800**	**0.759**	**0.464**	**3.1M**

Table 6. Final unseen test comparison of YOLO nano segmentation models.

Model / Run ID	Model	Optimizer	Image Size	Learning Rate	Weight Decay	Seg mAP50–95
E05	YOLO11n-seg	AdamW	800	3e-4	1e-4	0.454
V8_E04	YOLOv8n-seg	AdamW	512	3e-4	1e-4	**0.470**
E11_musgd	YOLO26n-seg	auto / MuSGD	800	5.4e-3	6.4e-4	0.464

parameter counts, making the comparison fair in terms of model capacity.

A notable observation is that YOLO26n-seg required more extensive tuning to reach its best result. The default AdamW settings underperformed compared to the native MuSGD optimizer, suggesting that YOLO26's pretrained weights are more tightly coupled to its original training recipe. In contrast, YOLOv8n-seg and YOLO11n-seg responded more predictably to standard AdamW tuning. This indicates that YOLO26 may require more careful setup but offers better peak performance when configured correctly.

I.2. Qualitative Results on Unseen Images

To compare all the models, what we did was test them on a set of unseen images that was not used in the training and validation. This test can tell us how well the model was trained with the best parameters that we were able to achieve in our training. Each model used the test images inside the dataset to see how well it could perform on images that it had not seen yet. Sometimes a model can remember patterns with some datasets, and if these models are going to be used in real-world practices, it is good to see if the model can handle and be accurate with unseen potholes. Potholes come in many different shapes and sizes, so this test will be good because potholes are very irregular, and this will put the model to the test and see if it can be accurate. Now we are looking at mAP50-95 because this metric tells us how accurate the outline is on the potholes.

When looking at the different models we saw with YOLO11, it performs decently with a segmentation mAP50-95 of 0.454. It had a weight decay of 1e-4, a learning rate of 3e-4, and an image size of 800. These values were selected because these parameters produced the best results in training, and we wanted to see how well they would perform in unseen data.

Moving on to YOLOv8, we saw that at image size 512, learning rate at 3e-4, and weight decay at 1e-4, we saw Seg mAP50-95 at 0.469. Same as for YOLO11, we used the best model that performed the best in our training, and here we saw that it produced a higher mAP50-95.

Going on to YOLO26, we saw that with an image size of 800, a learning rate of 5.4e-3, and a weight decay of 6.4e-4, we saw a Seg mAP50-95 of 0.464. While being lower than YOLOv8, it still performed very well compared to the training results that were discussed earlier. Now this was very interesting to see because our YOLOv8, compared to our YOLO26, performed better. YOLO26 is newer and is improved from the previous models, but there is a reason why YOLOv8 performed better.

When we look at the parameters, we see that YOLOv8 used an image size of 512 compared to the image size of 800 with our other models. This is very important because the image size does is it affects the resolution, and this affects how much detail the model can see. Having a low-end image size gives us less detail to see in an image, and with a high image size like 800, we are seeing more detail compared to a 512 image size. Since YOLOv8 had a 512 image size, it did not need to worry too much about detail. As for YOLO26, it does since we increased the image size to 800, so it is seeing more detail, and since potholes are irregular, the model will struggle a bit. That is why we see YOLOv8 perform better than YOLO26; it is because of the image size. If we were to run YOLOv8 at an 800 image size, the model will most likely perform lower because of all the details the model will see. Smaller image size reduces the amount of detail being seen but will train better. The moeel will struggle with harder images where detail is important and will most likely perform poorly.

J. Limitations and Future Works

J.1. Limitations

We did have some limitations when doing this project, and this was mainly due to the hardware we had. With some of our hardware not having a lot of RAM or memory, we were limited on some of our parameters, like our batch size, which had to be lowered because memory would run out. Also, having more time to get better results, because when running higher epochs with our limited hardware, the training would take a huge amount of time. Since our hardware was not the greatest, the training would take hours to complete.

J.2. Future Work

Some of the future work we would like to do is maybe train with more epochs to see if we could see better results. With more epochs, the model would be able to train more, and perhaps some of the parameters we change would receive better results. We can also upgrade our hardware to a stronger GPU, which can help out with the memory usage. Also, change more parameters and not just three parameters. See if we can find better learning rates, image size, or weight decay values for our model to be better. There are many more things that we can do, but these are some of the

main things that come to mind when thinking about what can help improve this project.

K. Conclusion

This project compared three YOLO nano segmentation models — YOLOv8n-seg, YOLO11n-seg, and YOLO26n-seg — on the task of pothole instance segmentation using the RoboFlow Roadvis dataset. Each model was fine-tuned from pretrained weights and evaluated under the same hyperparameter tuning strategy, adjusting learning rate, image size, and weight decay. The best configurations were then used in final training runs evaluated on an unseen test split.

All three models demonstrated the ability to segment potholes, with segmentation mAP50–95 scores above 0.45 after tuning. YOLO26n-seg achieved the highest final test score of 0.464 seg_mAP50–95 after tuning with the native MuSGD optimizer, confirming that using the optimizer consistent with the model's pretraining recipe is important for fine-tuning performance. YOLO11n-seg showed strong performance with standard AdamW tuning, while YOLOv8n-seg performed competitively despite being the oldest architecture.

A key finding from this project is that image size had a large impact on segmentation quality across all models. Larger image sizes generally improved mask accuracy, particularly for potholes with irregular boundaries. Weight decay showed minimal impact at 10-epoch fine-tuning, while learning rate required careful selection to avoid instability or slow convergence.

The results suggest that newer YOLO architectures like YOLO26 can offer improved segmentation performance on small, irregular objects such as potholes, but may require more knowledge of their native training settings to unlock their full potential. Future work could include training with more epochs, combining the best hyperparameters from all phases into a unified search, and testing on additional road surface datasets to evaluate generalization across different environments.

References

[1] Ultralytics. Yolov8. https://docs.ultralytics.com/models/yolov8/, 2023. Accessed: May 07, 2026. 11

[2] Ultralytics. Yolov8 vs. yolov9: A comprehensive technical comparison of real-time object detectors. https://docs.ultralytics.com/compare/yolov8-vs-yolov9/#yolov9-programmable-gradient-information, 2025. Accessed: May 07, 2026. 11

[3] A. Timilsina. Yolov8 architecture explained! https://abintimilsina.medium.com/yolov8-architecture-explained-a5e90a560ce5, 2024. Accessed: May 07, 2026. 11

[4] arXiv. What is yolov8: An in-depth exploration of the internal features of the next-generation object detector. https://arxiv.org/html/2408.15857v1, 2015. Accessed: May 07, 2026. 11

[5] IBM. Convolutional neural networks. https://www.ibm.com/think/topics/convolutional-neural-networks, 2021. Accessed: May 07, 2026. 11

[6] AI21. What is a convolutional neural network (cnn)? https://www.ai21.com/glossary/foundational-llm/convolutional-neural-network/, 2025. Accessed: May 07, 2026. 11

[7] arXiv. Comparing yolov11 and yolov8 for instance segmentation of occluded and non-occluded immature green fruits in complex orchard environment. https://arxiv.org/html/2410.19869v3#S2, 2022. Accessed: May 07, 2026. 11

[8] R. Sapkota, C. R. Harsha, A. Sharda, and M. Karkee. Yolo26: Key architectural enhancements and performance benchmarking for real-time object detection. https://arxiv.org/abs/2509.25164, 2025. Accessed: May 07, 2026. 11

[9] S. Rai. Roadvis segmentation dataset. https://universe.roboflow.com/sankritya-rai-cldft/roadvis-segmentation, 2023. Accessed: May 08, 2026. 11

[10] arXiv. Ultralytics yolo evolution: An overview of yolo26, yolo11, yolov8, and yolov5 object detectors for computer vision and pattern recognition. https://arxiv.org/html/2510.09653v1, 2019. Accessed: May 08, 2026. 11

[11] Ultralytics. Yolo11. https://docs.ultralytics.com/models/yolo11/. Accessed: May 08, 2026. 11

[12] Ultralytics. Instance segmentation. https://docs.ultralytics.com/tasks/segment/. Accessed: May 08, 2026. 11

[13] Ultralytics. Performance metrics deep dive. https://docs.ultralytics.com/guides/yolo-performance-metrics/. Accessed: May 08, 2026. 11

[14] Ultralytics. Model training with ultralytics yolo. https://docs.ultralytics.com/modes/train/. Accessed: May 08, 2026. 11

[15] Ultralytics. Yolo11 segmentation model yaml. https://raw.githubusercontent.com/ultralytics/ultralytics/main/ultralytics/cfg/models/11/yolo11-seg.yaml. Accessed: May 08, 2026. 11

[16] U. Ortega. Official deep learning final. https://github.com/ulyses97/Official-Deep-Learning-Final, 2026. Accessed: May 08, 2026. 11

[17] R. Khanam and M. Hussain. Yolov11: An overview of the key architectural enhancements. https://arxiv.org/html/2410.17725v1, 2015. Accessed: May 08, 2026. 11

[18] Ultralytics. Ultralytics yolo26. https://docs.ultralytics.com/models/yolo26, 2025. Accessed: May 12, 2026. 11

[19] P. Hidayatullah and R. Tubagus. Yolo26: A comprehensive architecture overview and key improvements. https://arxiv.org/abs/2602.14582, 2026. Accessed: May 12, 2026. 11